

JavaScript Operators

Operators are special symbols or keywords that perform operations on **operands** (values or variables).

For example:

```
let x = 5 + 3; // '+' is an operator, 5 and 3 are operands
```

JavaScript operators are used to perform **calculations**, **comparisons**, **assignments**, **logic checks**, and more.

1. Types of Operators in JavaScript

JavaScript has several types of operators:

- 1. Arithmetic Operators
- 2. Assignment Operators
- 3. Comparison Operators
- 4. Logical Operators
- 5. Bitwise Operators
- 6. String Operators
- 7. Ternary (Conditional) Operator
- 8. Type Operators

1

Arithmetic Operators

Used to perform **mathematical calculations**.

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15

Operator	Description	Example	Result
/	Division	6 / 2	3
%	Modulus (Remainder)	7 % 3	1
**	Exponentiation	2 ** 3	8
++	Increment (adds 1)	x++	increases by 1
--	Decrement (subtracts 1)	x--	decreases by 1

Example:

```
let x = 10;
x++;
console.log(x); // 11
```

2 Assignment Operators

Used to **assign values** to variables.

Operator	Example	Same As	Meaning
=	x = 5	—	Assign 5 to x
+=	x += 3	x = x + 3	Add and assign
-=	x -= 2	x = x - 2	Subtract and assign
*=	x *= 2	x = x * 2	Multiply and assign
/=	x /= 2	x = x / 2	Divide and assign
%=	x %= 3	x = x % 3	Modulus and assign
**=	x **= 2	x = x ** 2	Exponentiate and assign

Example:

```
let x = 5;
x += 10; // x = x + 10
console.log(x); // 15
```

3 Comparison Operators

Used to **compare two values** and return a **Boolean** (`true` or `false`).

Operator	Description	Example	Result
<code>==</code>	Equal to (loose equality)	<code>5 == "5"</code>	<code>true</code>
<code>===</code>	Strict equal to (checks value & type)	<code>5 === "5"</code>	<code>false</code>
<code>!=</code>	Not equal to	<code>5 != 3</code>	<code>true</code>
<code>!==</code>	Strict not equal	<code>5 !== "5"</code>	<code>true</code>
<code>></code>	Greater than	<code>7 > 3</code>	<code>true</code>
<code><</code>	Less than	<code>7 < 3</code>	<code>false</code>
<code>>=</code>	Greater than or equal	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	Less than or equal	<code>5 <= 3</code>	<code>false</code>

Example:

```
let age = 18;
console.log(age >= 18); // true
```

 Note:

Always prefer `===` and `!==` to avoid **type coercion confusion**.

4 Logical Operators

Used to **combine or invert** Boolean values.

Operator	Description	Example	Result				
<code>&&</code>	Logical AND	<code>true && false</code>	<code>false</code>				
<code>,</code>		<code>,</code>	Logical OR	<code>`true</code>		<code>false`</code>	<code>true</code>
<code>!</code>	Logical NOT	<code>!true</code>	<code>false</code>				

Example:

```
let age = 20;
let hasVoterCard = true;

if (age >= 18 && hasVoterCard) {
  console.log("Eligible to vote");
}
```

✔ Notes:

- && → all conditions must be true
- || → at least one condition must be true
- ! → inverts a Boolean value

5 Bitwise Operators (Advanced)

Operate directly on **binary numbers** (bits).

Operator	Name	Example	Result		
&	AND	5 & 1	1		
	OR	5 1	5	1	5
^	XOR	5 ^ 1	4		
~	NOT	~5	-6		
<<	Left shift	5 << 1	10		
>>	Right shift	5 >> 1	2		

Example:

```
console.log(5 & 1); // 1
```

6 String Operators

Used for **concatenating (joining)** strings.

Operator	Description	Example	Result
+	Concatenate strings	"Hello" + " World"	"Hello World"
+=	Add and assign	text += "!"	"Hello!"

Example:

```
let firstName = "Ismail";
let lastName = "Bankole";
console.log(firstName + " " + lastName);
```

✓ Tip:

Prefer **template literals** for readability:

```
console.log(`My name is ${firstName} ${lastName}`);
```

7 Ternary (Conditional) Operator

A shorthand way to write an `if...else` statement.

```
condition ? valueIfTrue : valueIfFalse;
```

Example:

```
let age = 20;
let status = age >= 18 ? "Adult" : "Minor";
console.log(status); // Adult
```

8 Type Operators

Used to check or convert data types.

Operator	Description	Example	Result
typeof	Returns data type	typeof 123	"number"

Operator	Description	Example	Result
instanceof	Checks if an object is an instance of a class	arr instanceof Array	true

Example:

```
let fruits = ["apple", "banana"];
console.log(typeof fruits); // object
console.log(fruits instanceof Array); // true
```

9 Operator Precedence

Operator precedence determines **which operator is evaluated first** in complex expressions.

Example:

```
let result = 5 + 3 * 2;
console.log(result); // 11 (Multiplication happens before addition)
```

✔ Precedence order (highest to lowest):

- 1. () Parentheses
- 2. ** Exponentiation
- 3. * , / , %
- 4. + , -
- 5. Comparisons (> , < , etc.)
- 6. Logical (&& , ||)
- 7. Assignment (= , += , etc.)

◆ Summary Table

Category	Examples	Description		
Arithmetic	+ , - , * , / , % , ++ , --	Math operations		

Category	Examples	Description		
Assignment	= , += , -= , *= , /=	Assign or update values		
Comparison	== , === , != , > , <	Compare values		
Logical	&& , `		, !`	Combine Boolean expressions
String	+ , +=	Join text		
Ternary	condition ? a : b	Short if...else		
Type	typeof , instanceof	Check or verify data types		